UNITÀ 2 · WEB SERVICE · LEZIONE 9

Un web service RESTful in Java

BiblioAPI prende vita: un server REST con le operazioni CRUD, scritto con il solo JDK, senza framework. Ogni byte che parte lo decidi tu.

5G TPSI HttpServer, HttpExchange, JSON, codici di stato, concorrenza

PER COMINCIARE

Se domani dovessi scrivere un'API in Java per un'azienda, con che cosa la scriveresti?

SONDAGGIO

- **A** Spring Boot
- **B** Jakarta EE, con Jakarta REST
- **C** Solo il JDK, come oggi
- **D** Me la farei scrivere da un'AI
- **E** Non saprei da dove iniziare

LO STESSO ENDPOINT, TRE MODI

I framework fanno con un'annotazione ciò che oggi scriviamo a mano

Spring Boot

```
@GetMapping("/libri/{id}")
Libro uno(@PathVariable
          int id) {…}
```

Il framework più usato nelle aziende Java.

Jakarta REST

```
@GET @Path("/libri/{id}")
Libro uno(@PathParam("id")
          int id) {…}
```

Lo standard di Jakarta EE, ex JAX-RS.

Solo JDK

```
server.createContext(
  "/api/libri", ex -> {
  // tutto a mano });
```

Niente magia: routing, JSON e codici sono codice nostro.

COME È FATTO IL SERVER

Dal socket al tuo metodo

HttpServer

Ascolta sulla porta 8000 e legge le richieste HTTP.

Contesto

Sceglie l'handler in base al prefisso del percorso: `/api/libri`.

Handler

Il tuo codice: guarda metodo e percorso e decide.

HttpExchange

Richiesta e risposta: header, corpo, codice.

> Un **Executor** decide quale thread esegue l'handler. Due client possono chiamare l'API nello stesso istante: il codice deve reggere richieste **concorrenti**.

IL PRIMO ENDPOINT

Quindici righe per un web service

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;

public class BiblioServer {
  public static void main(String[] args) throws Exception {
    HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
    server.createContext("/api/ping", ex -> {
      byte[] dati = "{\"stato\":\"ok\"}".getBytes(StandardCharsets.UTF_8);
      ex.getResponseHeaders().set("Content-Type", "application/json");
      ex.sendResponseHeaders(200, dati.length);   // prima gli header…
      try (var os = ex.getResponseBody()) { os.write(dati); }  // …poi il corpo
    });
    server.start();
  }
}
```

AL COMPUTER · 15 minuti

Accendi BiblioAPI

1. Crea la cartella **biblioapi** e dentro il file **BiblioServer.java** con il codice della slide precedente.
2. Dal terminale, nella cartella: `java BiblioServer.java`. Il terminale resta occupato: è il server.
3. Apri nel browser `http://localhost:8000/api/ping`.
4. In un secondo terminale: `curl -i http://localhost:8000/api/ping`.
5. Prova anche `/api/pong`. Poi aggiungi una riga che stampi "BiblioAPI pronta" all'avvio.

Controllo: il browser mostra `{"stato":"ok"}`. curl mostra `HTTP/1.1 200 OK`, `Content-type: application/json` e `Content-length: 14`. Per `/api/pong`: **404**, perché nessun contesto corrisponde.

QUIZ PER CHI LEGGE LA DOCUMENTAZIONE

DELETE riuscita: 204 No Content. Che cosa passi come lunghezza?

QUIZ

A `ex.sendResponseHeaders(204, 0)`

B `ex.sendResponseHeaders(204, null)`

C `ex.sendResponseHeaders(204, -1)`

D Nessuna: con 204 non si chiama

IL MODELLO E LE RISPOSTE

Un record, una mappa, un metodo che risponde

```
record Libro(int id, String titolo, String autore, int anno) {
  String toJson() {
    return "{\"id\":%d,\"titolo\":\"%s\",\"autore\":\"%s\",\"anno\":%d}"
        .formatted(id, esc(titolo), esc(autore), anno);
  }
}
static final Map<Integer, Libro> libri = new ConcurrentHashMap<>();
static final AtomicInteger prossimoId = new AtomicInteger(1);

static void rispondi(HttpExchange ex, int codice, String json) throws IOException {
  byte[] dati = json.getBytes(StandardCharsets.UTF_8);
  ex.getResponseHeaders().set("Content-Type", "application/json; charset=utf-8");
  ex.sendResponseHeaders(codice, dati.length);
  try (OutputStream os = ex.getResponseBody()) { os.write(dati); }
}
```

PUZZLE

Abbina ogni metodo di HttpExchange a ciò che restituisce

getRequestMethod()	---	Un `InputStream` con il JSON inviato dal client
getRequestURI().getPath()	---	`"/api/libri/3"`
getRequestURI().getQuery()	---	Gli header da preparare prima di rispondere
getRequestHeaders().getFirst(...)	---	Il valore di un header, come `application/json`
getRequestBody()	---	`"autore=Calvino"`
getResponseHeaders()	---	`"PUT"`

Metodo + percorso = che cosa fare

```
static void gestisci(HttpExchange ex) throws IOException {
  String metodo = ex.getRequestMethod();
  String[] parti = ex.getRequestURI().getPath().split("/");  // ["", "api", "libri", "3"]
  try {
    if (parti.length == 3) {                                  // /api/libri
      switch (metodo) {
        case "GET"  -> rispondi(ex, 200, elenco());
        case "POST" -> crea(ex);
        default     -> rispondi(ex, 405, errore("metodo non consentito"));
      }
    } else if (parti.length == 4) {                           // /api/libri/{id}
      Libro l = libri.get(Integer.parseInt(parti[3]));
      if (l == null) { rispondi(ex, 404, errore("libro non trovato")); return; }
      // switch su GET, PUT, DELETE…
    } else rispondi(ex, 404, errore("risorsa non trovata"));
  } catch (NumberFormatException e) { rispondi(ex, 400, errore("id non valido")); }
}
```

AL COMPUTER · 30 minuti

Leggere i libri: GET

1. Aggiungi a BiblioServer il record, la mappa, `rispondi`, `errore` ed `elenco`.
2. All'avvio inserisci i tre libri di esempio con id 1, 2, 3.
3. Scrivi `gestisci` con i rami GET e registralo sul contesto `/api/libri`.
4. Riavvia il server (Ctrl+C e di nuovo `java BiblioServer.java`).
5. Prova `/api/libri`, `/api/libri/2`, `/api/libri/9`, `/api/libri/abc`.

Controllo: `/api/libri` → 200 e un array di 3 oggetti. `/api/libri/2` → 200 e `"autore":"Italo Calvino"`. `/9` → **404** con `{"errore":…}`. `/abc` → **400**.

CREARE UNA RISORSA

POST: leggi, valida, salva, rispondi 201

```
static void crea(HttpExchange ex) throws IOException {
  String corpo = new String(ex.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
  String titolo = campo(corpo, "titolo"), autore = campo(corpo, "autore");
  Integer anno = numero(corpo, "anno");
  if (titolo == null || autore == null || anno == null) {
    rispondi(ex, 400, errore("servono titolo, autore e anno")); return;
  }
  int id = prossimoId.getAndIncrement();
  Libro l = new Libro(id, titolo, autore, anno);
  libri.put(id, l);
  ex.getResponseHeaders().set("Location", "/api/libri/" + id);
  rispondi(ex, 201, l.toJson());
}
```

PUZZLE

Una richiesta POST arriva al server: metti in ordine

1. Legge il corpo e ne estrae titolo, autore e anno
2. Se manca un campo risponde 400 e si ferma
3. Il contesto `/api/libri` passa lo scambio all'handler
4. L'handler legge metodo e percorso e sceglie `crea`
5. Imposta gli header Location e Content-Type
6. Invia il codice 201, scrive il JSON e chiude lo stream
7. Prende un nuovo id e salva il libro nella mappa

AL COMPUTER · 35 minuti

CRUD completo: POST, PUT, DELETE

```
libro.json
{"titolo":"Il nome della rosa","autore":"Umberto Eco","anno":1980}
comandi (in PowerShell scrivi curl.exe)
curl -i -X POST -H "Content-Type: application/json" -d @libro.json http://localhost:8000/api/libri
curl -i -X DELETE http://localhost:8000/api/libri/4
```

1. Scrivi `campo`, `numero`, `crea` e `aggiorna`; completa i rami PUT e DELETE.
2. Crea **libro.json** nella cartella del server e prova i comandi.
3. Prova anche un PUT su `/api/libri/4` e un POST senza anno.

Controllo: POST → **201** e `Location: /api/libri/4`; ripetuto crea il 5. PUT → 200. DELETE → **204** senza corpo, poi GET `/4` → **404**. Senza anno → **400**.

QUIZ SULLA CONCORRENZA

Con una HashMap e un `int` contatore, due POST arrivano nello stesso istante. Che cosa può succedere?

QUIZ

A Due libri diversi possono ricevere lo stesso id

B Niente: HttpServer serve sempre una richiesta per volta

C Il secondo client riceve 409 Conflict

D La JVM se ne accorge e lancia un'eccezione

MENTI CHE CI HANNO PRECEDUTO

Prima il codice, poi le discussioni

> **Parlare costa poco. Fammi vedere il codice.**

Linus Torvalds, creatore del kernel Linux, in un messaggio alla mailing list del kernel, 25 agosto 2000



Che cosa controllare prima di fidarsi

- I codici: 201 e Location, 204, 400, 404, 405.
- La validazione dell'input, prima di salvare.
- L'escape del JSON e dei caratteri speciali.
- Le strutture dati condivise tra thread.
- Nessun segreto o password scritti nel codice.



BIGLIETTO D'USCITA

Completa prima di uscire

La classe del JDK che ascolta le richieste HTTP si chiama _____.

Per rispondere senza corpo si passa come lunghezza il valore ____.

Dopo un POST riuscito il server invia l'header _____ con l'URI del nuovo libro.

Un metodo non previsto su una risorsa riceve il codice _____.

Per un contatore condiviso tra thread si usa _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- HttpServer, contesti, handler, HttpExchange
- Prima header e codice, poi corpo, poi chiusura
- Routing su metodo e segmenti del percorso
- POST: valida, salva, 201 + Location
- Codici d'errore con un JSON che spiega
- Richieste concorrenti: ConcurrentHashMap, AtomicInteger

> **All'esame**: per il lato server di un progetto basta un elenco degli endpoint e lo pseudocodice di uno di essi (leggi, valida, salva, rispondi). Saper motivare i codici di stato fa la differenza all'orale.

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso.